

Rust Armor Separation Kernel » Port to RISC-V SMP

Marco Ferraresi

Edoardo Figini

September 2, 2026

Contents

1	Project data	1
2	Project description	1
2.1	Design and implementation	2
3	Project outcomes	3
3.1	Concrete outcomes	3
3.2	Learning outcomes	4
3.3	Existing knowledge	4
3.4	Problems encountered	5
4	Honor Pledge	5

1 Project data

- Project supervisor: Prof. Vittorio Zaccaria
- Group composition:

Last and first name	Person code	Email address
Ferraresi Marco	10764701	marco2.ferraresi@mail.polimi.it
Figini Edoardo	10771695	edoardo.figini@mail.polimi.it

- Development tasks were **not** subdivided: the two of us worked on the whole project together, in shared pair-programming sessions, from the first “hello world” on the board up to the final test suite.
- Links to the project source code:
 - Repository: <https://github.com/polimi-aos-lab/aSK>
 - Branch of this project: `milkv-megrez`

2 Project description

The goal of this project is to port aSK to the Milk-V Megrez board, a new architecture featuring the RISC-V Hypervisor Extension and Memory Management Unit (MMU).

aSK (*Armor Separation Kernel*) is a small type-1 hypervisor written in Rust. It is a *separation kernel*: its only job is to run several execution environments on one chip and guarantee that they cannot interfere with each other.

Our project was to make it run on a Milk-V Megrez board, based on the ESWIN EIC7700X SoC (four harts, ISA rv64imafdch) and to make it use the **native virtualization support** of RISC-V instead of the hand-made approach: the hypervisor runs in HS-mode on top of standard OpenSBI firmware, the guests run in VS/VU-mode, and isolation is enforced by G-stage page tables, the second address translation stage that only the hypervisor controls.

We faced two main challenges on this project:

- The **privilege level**: from M-mode to HS-mode, which changes the control registers, the instruction used to enter a guest, and the way timers and inter-processor interrupts are obtained.
- The **memory model**: from PMP regions to page tables with two translation stages.

This project is important for the AOS course because we got to apply the concepts about virtualization and memory protection that we learned in class to a real board and kernel.

2.1 Design and implementation

aSK is organised in three layers: an architecture-agnostic *kernel*, a *VHAL* layer of traits that states what an architecture must provide, and the *arch* implementations. Porting aSK therefore means implementing about ten traits (*CpuOps*, *VmmOps*, *MpuOps*, *InterruptControllerOps*, ...) without touching the kernel. All our code lives in a new module, `sk/src/arch/riscv64h/`, plus the board crate `boards/milkv-megrez-bxxx/`.

The module is selected by a **Cargo feature** (`hsmode` against `mmode`) rather than by a custom target, since Spike and the Megrez share the same target flag.

Memory model

The first challenge we faced is how to make the region-based memory API survive on top of page tables.

The kernel talks to an MPU-shaped interface, so each method was reimplemented in terms of translation: hypervisor regions become entries in an Sv39 identity map rooted in `satp`, VM regions become entries in the per-VM Sv39x4 G-stage rooted in `hgsatp`, and enabling protection means switching `satp` from Bare to Sv39.

The hypervisor identity map covers the first 4 GiB with four 1 GiB superpages, which keeps the page-table footprint to a single table; page tables come from static pools in the BSS. Each VM gets its own G-stage, restricted to its own RAM window and tagged with its own VMID.

Virtual Machine Monitor

From a guest's point of view aSK is the firmware: every `ecall` from VS-mode traps into HS-mode and is served by our SBI implementation, with timer requests forwarded to the real OpenSBI underneath. Devices a guest does not own are given to it by *trap-and-emulate*: the MMIO address is left out of the G-stage, the resulting guest-page fault is decoded from `htval` and `htinst`, and an NS16550 UART is emulated in software.

Since `htinst` is not reliable in RISC-V H-extension V0.6, the hypervisor has to decode the instruction from the guest PC. A feature flag determines when the hypervisor runs on hardware that implements V0.6 (Section 2.1).

Multi-hart boot

Multi-hart bring-up follows the OpenSBI model, where only one hart starts and the others must be woken through the HSM extension. This required a second assembly entry point for secondary harts that skips the BSS clear (the master has already filled global state by then) and a runtime election of the master hart, because the boot hart is chosen by the firmware and is not predictable.

VM isolation

Isolation is demonstrated by running two VMs, in disjoint RAM windows, on different harts, each with its own G-stage and VMID, while one of them deliberately reads the other's memory. The resulting fault is not treated as a fatal error: it is logged and **only the offending VM is stopped**, which is precisely the semantics that distinguishes a separation kernel from an ordinary hypervisor.

Finally, VMs can own **real** devices: a VM declares an interrupt in its static configuration, the kernel enables that PLIC source on the context of the hart hosting it, and when the device fires, the interrupt is claimed in HS-mode, matched against the routing table and injected into the owning guest as a virtual external interrupt through `hvip`.

RISC-V H-extension V1.0 vs V0.6

Milk-V Megrez implements the Hypervisor Extension in a pre-ratified version (V0.6).

The differences affect CSR behaviour and presence

Feature	Issue	Solution
<code>htinst</code>	Present but undocumented; cannot depend on it	Feature flag and instruction fetched from guest PC
<code>vstimecmp</code>	Not present	This was not an issue because timer are set using the SBI layer
<code>hstatus.VGEIN</code>	In V0.6 not implemented, reserved on megrez	This was not an issue because we do not use guest external interrupts, we just needed to write 0 during boot
<code>htval</code>	Present but undocumented	No solution, we have to verify on the hardware if some issues arise
<code>henvcfg</code>	Not present	This was not an issue because we do not touch it
<code>hgeie / hgeip</code>	underspecified in V0.6	This was not an issue because we do not touch them

3 Project outcomes

3.1 Concrete outcomes

The deliverable is the `milkv-megrez` branch of the aSK repository:

- A new architecture module, `sk/src/arch/riscv64h/`, implementing the full VHAL contract for HS-mode: boot and per-hart setup, CPU operations, memory management on Sv39 page tables, the VMM with its hypervisor vector table and SBI dispatcher, the PLIC driver and the interrupt controller with virtual interrupt injection and physical-IRQ routing.
- A dual platform configuration in `sk/src/arch/riscv64h/boards/milkv_megrez.rs`: the real EIC7700X one, and a QEMU one that is reproducible on any machine. The QEMU configuration is used in the test suite.
Commit `a674175` "[milkv-megrez] EIC7700X platform config"
- The board crate `boards/milkv-megrez-bxxx/`, where (in `main.rs`) the two VMs are declared statically with their memory, their emulated UART and the physical interrupt one of them owns.
Commit `0de73bd` "[milkv-megrez] multi-hart multi-VM boot flow"
- Integration into the repository test infrastructure: a Docker image with QEMU, OpenSBI and U-Boot, and two smoke-test scenarios:

- one covering the boot chain up to the point where the guests start.
- one covering the VM layer (MMIO emulation, interrupt injection, physical IRQ routing and end-of-interrupt, isolation violation and clean shutdown)

run with `make test-riscv-h`.

Commit 92be7d8 "[milkv-megrez] integrated milkv-megrez in test suite"

All eight phases of the roadmap agreed with the supervisor are complete and verified on the emulated platform, and the first four (boot, console, CPU, MMU with the identity map) were also validated on the physical board.

What remains is re-running the complete integrated flow on the hardware with the real platform configuration; this is where the differences of the EIC7700X implementation of the Hypervisor Extension, which predates the ratified specification, are expected to show up.

3.2 Learning outcomes

Tools we learned to use and knowledge we gained during this project:

- Rust programming language: we both started this project with a surface level knowledge of Rust. This project gave us both the opportunity to explore the language (trait-implementation paradigm and safety oriented approach) and the ecosystem (Cargo, crates.io, Rustup, etc.) in depth.
- Privileged RISC-V: we were already familiar with the user-level RISC-V ISA from previous courses, this project gave us the opportunity to explore how RISC-V works beyond the simple pipeline we had studied (control status registers, traps, interrupts, memory management).
- QEMU: we learned how to configure QEMU to emulate a RISC-V board like Milk-V Megrez.
- Virtualization: we got to explore hands-on how hardware assisted virtualization enables virtual machines to run efficiently at near-native speed.

We learned how the hypervisor does not actively control VMs, but handles exceptions raised by the hardware, allowing for simpler design and better performance.

We took this project as an opportunity to enrich our knowledge of low-level systems, a subject we are both interested in and wanted to explore more. Specifically, Marco was really interested in learning about Embedded Rust, since it is becoming a more and more popular language with an active community and a growing ecosystem. Edoardo was more interested in developing a low-level system and writing software that interacts with the hardware. Along with that, he wanted to become more familiar with the RISC-V architecture as it is growing in popularity and it's exiting the niche academic world into commercial applications. Both of us will bring the knowledge we gained in this project into our future personal and professional projects.

3.3 Existing knowledge

Some courses we followed have helped us doing this project:

- Embedded Systems: theoretical concepts in line with AOS course, but with a focus on firmware and hardware interaction: this helped us as this project was focused on bridging the gap between the hardware and the higher level software (the hypervisor).
- Sensor Systems and Design of Hardware Accelerators: in these courses we had to interact with a board using only UART for debugging without fancy tools like gdb as we used in high level software development.

3.4 Problems encountered

- **Total silence at the first boot on the board.** The binary was loaded and started, and nothing happened: no output, no error. The cause was the firmware boot protocol: the U-Boot `go` command is a raw jump and does not set the registers that carry the hart identifier, so our code read garbage, concluded it was a secondary hart, and waited forever for a master that would never arrive. The solution came by looking at the U-Boot documentation online and, after a lengthy trial and error process, we concluded the correct command should have been `bootelf`. This meant that U-Boot accepts elf binaries, so we didn't have to provide the raw binary.
- **The boot hart is not predictable.** Our first multicore code assumed hart 0 would be the master. The firmware picks a different hart on different runs, which deadlocked the rendez-vous. We replaced the assumption with a runtime election, using an atomic compare-and-exchange on a variable placed in initialised data so that it survives the zeroing of the BSS.
- **Unsynchronized console.** With several harts printing to a serial port with no locking, output lines from different cores are interleaved character by character. This is cosmetic when reading logs by eye, but it made automated test patterns unreliable. We synchronized the hypervisor's own output with a per-line lock, which also made the interesting events assertable in the test suite. Guest output is still interleaved, since the locking mechanism we implemented lives in the hypervisor and is not available to the guest.

None of us had encountered these specific problems before. What they have in common is that the system does not show what is wrong: there is no error message, no exception and often no output at all, so we could not rely on a traditional debugging tools. We had to understand the behaviour of our code through rudimentary "printf" debugging.

4 Honor Pledge

We pledge that this work was fully and wholly completed within the criteria established for academic integrity by Politecnico di Milano (Code of Ethics and Conduct) and represents our original production, unless otherwise cited.

We also understand that this project, if successfully graded, will fulfill part B requirement of the Advanced Operating System course and that it will be considered valid up until the AOS exam of Sept. 2026.

Marco Ferraresi, Edoardo Figini